HONEYWELL HACKATHON | FINAL SUBMISSION

GradeSense Deployment Guide

Local, Docker, Render, WebSocket, and operational deployment

GRADE	SENSE	AI

Explainable predictions, constraint-aware recommendations, and real-time operations for safer, faster paper grade transitions.

Prepared from the validated GradeSense v0.2.0 source repository and live application.

Deployment Overview

The reference system supports local developer startup, Docker Compose, and a Render blueprint. Production operation requires externally managed TLS, identity, secrets, backups, and telemetry retention.

Backend Setup

GradeSense | Honeywell Hackathon Submission

- Create a Python 3.11+ virtual environment inside backend/.
- Install the project: python -m pip install -e .
- Copy backend/.env.example to backend/.env and set environment-specific values.
- Run migrations: alembic upgrade head
- Start: uvicorn app.main:app --host 0.0.0.0 --port 8000
- Verify: GET /health and GET /admin/health

Frontend Setup

GradeSense | Honeywell Hackathon Submission

- Install Node.js 20+ and run npm install in frontend/.
- Set VITE_API_URL=http://localhost:8000 and VITE_WS_URL=ws://localhost:8000.
- Development: npm run dev -- --host 0.0.0.0 --port 5173
- Production build: npm run build; serve frontend/dist through Nginx.

Docker

GradeSense | Honeywell Hackathon Submission

Docker Compose starts PostgreSQL, the FastAPI backend, and the Nginx-served frontend. Model and dataset directories are mounted read-only and PostgreSQL uses a durable named volume.

- Validate: docker compose config --quiet
- Start: docker compose up --build -d
- Observe: docker compose ps; docker compose logs -f backend
- Stop: docker compose down
- Do not add --volumes unless database history should be intentionally deleted.

Render Deployment

GradeSense | Honeywell Hackathon Submission

The repository render.yaml defines the deployment blueprint. Connect the repository in Render, create services from the blueprint, provision the database, and set all secret variables in the Render dashboard.

Use the backend public URL for VITE_API_URL and the corresponding wss:// URL for VITE_WS_URL. Confirm CORS contains the exact frontend origin.

- Backend build installs the Python project and applies Alembic migrations before serving.
- Frontend build compiles the Vite bundle.
- Health checks must target /health for the API and /healthz for Nginx.

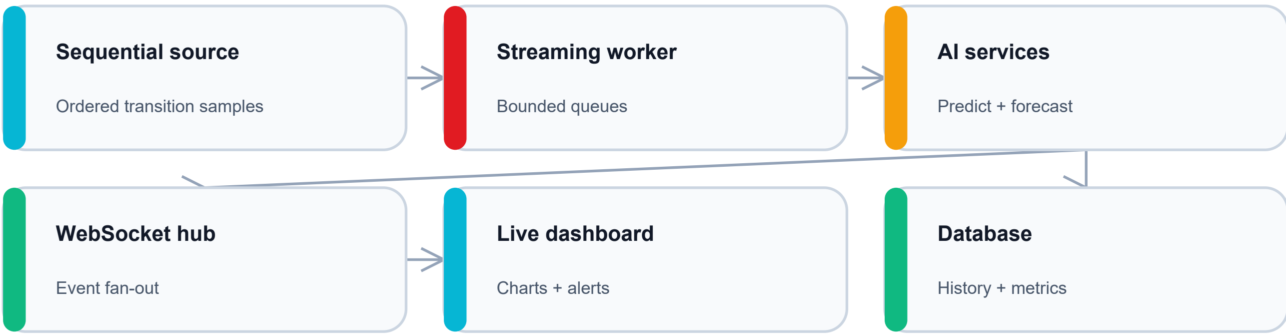
Environment Variables

GradeSense | Honeywell Hackathon Submission

Variable	Purpose
GRADESENSE_ENVIRONMENT	development, test, staging, production
GRADESENSE_DATABASE_URL	SQLAlchemy database URL
GRADESENSE_CORS_ORIGINS	JSON list of allowed origins
GRADESENSE_DATASET_PATH	Snapshot dataset path
GRADESENSE_SEQUENTIAL_DATASET_PATH	Ordered transition dataset
GRADESENSE_MODEL_PATH	Snapshot model artifact
GRADESENSE_FORECAST_MODEL_PATH	Forecast model artifact
GRADESENSE_STREAM_INTERVAL_SECONDS	Replay cadence
GRADESENSE_LOG_LEVEL	Structured logging level
VITE_API_URL	Browser-visible REST base URL
VITE_WS_URL	Browser-visible WebSocket base URL

WebSocket Architecture

GradeSense | Honeywell Hackathon Submission



WebSocket event envelope

```
{ "event": "prediction", "timestamp": "ISO-8601", "data": { ... } }
```

API Endpoints

GradeSense | Honeywell Hackathon Submission

- REST documentation: /docs, /redoc, /openapi.json

- Live sockets: /ws/live and /ws/dashboard
- Liveness: /health
- Deep health: /admin/health
- Current stream: /stream/status, /metrics/live, /metrics/rolling
- Core inference: /predict, /forecast, /forecast/simulate, /interventions/recommendations

Reverse Proxy and Scaling

- Preserve Upgrade and Connection headers for WebSockets.
- Use idle timeouts longer than the heartbeat interval.
- A single backend replica is the reference because stream state is process-local.
- Multi-replica operation requires a shared broker, coordinated ingestion, centralized metrics/rate limiting, and WebSocket fan-out or sticky sessions.

Verification Checklist

GradeSense | Honeywell Hackathon Submission

- All four model/data artifacts exist and are readable.
- Database migrations are at head.
- /health and /admin/health return healthy.
- Frontend can call the API with no CORS error.
- A browser receives heartbeat and prediction WebSocket events.
- Model registry shows active snapshot and forecast artifacts.
- Backups, TLS, identity, logs, and secrets are configured before production traffic.